Fully Integer-Quantized Thermal Image Super-Resolution on Low-Cost Microcontrollers

Adrian-Gabriel Diaconu Andrei-Alexandru Ulmămei
Faculty of Electronics, Telecommunications and Information Technology
National University of Science and Technology POLITEHNICA Bucharest
Bucharest, Romania
adrian.diaconu1305@stud.etti.upb.ro, alexandru.ulmamei@upb.ro

***Abstract*—Low-cost thermal sensors such as the MLX90640 deliver only 32×24 px frames, which limits their use in presence detection and human–machine interfaces. We show that convolutional super-resolution (SR) can run *entirely on the microcontroller that reads the sensor*. Five compact SR networks (1.6 k–159 k parameters) are redesigned around a restricted operator vocabulary (`CONV_2D`, `ADD`, `DEPTH_TO_SPACE`) so that post-training quantization yields *fully* integer INT8 models (int8 weights, activations and I/O). Full integer form is a hard requirement of embedded TFLite Micro runtimes, which reject hybrid graphs at runtime. On a FLIR-derived thermal validation set, all quantized models surpass bicubic interpolation (30.56 dB) by 0.2–2.0 dB, at a quantization penalty of only 0.17–0.64 dB. We benchmark all models on two low-cost MCUs: an ESP32 (240 MHz, no PSRAM) and a Raspberry Pi Pico (RP2040, 133 MHz), reporting measured latency (102 ms–10.0 s per frame), tensor-arena memory (19–129 KB), flash footprint (5–185 KB) and estimated energy. The smallest model sustains the sensor's native 8 Hz refresh rate on both boards. Its on-device output matches the PC reference within ±4 int8 LSB, so desktop-side quality carries over to the device.**

***Index Terms*—super-resolution, thermal imaging, quantization, TinyML, microcontrollers, TensorFlow Lite Micro**

I. Introduction

Far-infrared thermal sensing is increasingly used for occupancy detection, touchless interfaces, and condition monitoring, because it is illumination invariant and privacy preserving. The price of a thermal pixel, however, remains orders of magnitude above that of a visible-light pixel: array sensors in the sub-10-euro class, such as the Melexis MLX90640 [1], provide only 32×24 thermopile pixels. Single-image super-resolution (SISR) offers a way to trade abundant, cheap compute for scarce, expensive optics. In the intended deployments, the only compute available is the microcontroller (MCU) already reading the sensor.

Running SISR on an MCU imposes a constraint that is stricter than the usual TinyML size budget: the Espressif esp-nn kernels used by TensorFlow Lite Micro (TFLM) [2] on the ESP32 *reject hybrid models at runtime*: graphs whose weights are int8 but whose activations remain float32 (the default outcome of dynamic-range post-training quantization). A deployable model must be *fully* integer: int8 weights, int8 activations, and int8 input/output tensors. In our experience this failure mode is silent until the first on-device `AllocateTensors()` call, which motivates designing for full-integer quantization from the start rather than converting trained models after the fact.

This paper makes two contributions. *First*, we show that five representative compact SR architectures (ESPCN [3] at three widths, FSRCNN [4], and a reduced EDSR [5]) can all be expressed in a three-operator vocabulary (`CONV_2D`, `ADD`, `DEPTH_TO_SPACE`) with int8-friendly substitutions that cost little accuracy, so that full-integer INT8 conversion succeeds *by construction*. *Second*, we present a complete cross-board benchmark of the resulting models on two popular low-cost MCUs, measuring latency, memory, flash, reconstruction quality, quantization penalty, and device-vs-host numerical fidelity, and reporting estimated energy per frame. All results, code, and firmware are reproducible from a single pipeline.

II. Related Work

Compact SISR networks emerged with SRCNN [6], which maps a bicubically upsampled image through three convolutions. FSRCNN [4] operates in low-resolution (LR) space and upsamples with a learned 9×9 stride-2 deconvolution; ESPCN [3] introduced the sub-pixel convolution (*pixel shuffle*), keeping all computation in LR space. EDSR [5] scaled residual networks to state-of-the-art quality; we use a strongly reduced variant. Integer-only inference was formalized by Jacob *et al.* [7]; TFLM [2] brought the int8 runtime to kilobyte-class devices. Prior thermal SR work targets larger inputs and GPU-class hardware; to our knowledge this is the first systematic study of *fully integer* SR at MLX90640-scale resolution (32×24 → 64×48) with measurements on commodity MCUs.

III. Full-INT8 SR Networks

A. INT8-friendly architecture modifications

All five networks are built from three TFLM operators with mature int8 kernels: `CONV_2D` (with fused ReLU), `ADD`, and `DEPTH_TO_SPACE`. Table I summarizes the deviations from the original architectures.

TABLE I
INT8-FRIENDLY MODIFICATIONS PER ARCHITECTURE.

Model	Modification vs. original
ESPCN [3]	Tanh → ReLU
ESPCN-Light / -Micro	— (width-reduced ESPCN, ReLU)
FSRCNN-ds [4]	PReLU → ReLU; 9×9 s2 deconv → 3×3 conv + `depth_to_space`
EDSR-Tiny [5]	residual scaling 0.1 folded into conv weights at export

TABLE II
FOOTPRINT AND RECONSTRUCTION QUALITY (219-PAIR VALIDATION SPLIT).

Model	Params	MMAC	Flash (KB)	PSNR_{f32} (dB)	SSIM_{f32}	$\Delta\text{PSNR}_{\text{int8}}$ (dB)	$\text{SSIM}_{\text{int8}}$
Bicubic	—	—	—	30.56	—	—	—
ESPCN-Micro	1.6 k	1.2	5.2	30.92	0.905	−0.17	0.899
ESPCN-Light	6.0 k	4.6	10.1	31.45	0.911	−0.64	0.892
FSRCNN-ds	10.1 k	7.6	20.8	31.61	0.913	−0.49	0.895
ESPCN	21.3 k	16.3	26.2	31.73	0.914	−0.28	0.903
EDSR-Tiny	158.7 k	121.4	185.2	32.83	0.927	−0.25	0.920

Three substitutions deserve justification. *Tanh → ReLU* (ESPCN): ReLU fuses into the convolution's int8 requantization step, whereas Tanh needs a separate lookup-table kernel; using one activation type across all models also keeps the firmware operator resolver minimal. *PReLU → ReLU and deconvolution → sub-pixel head* (FSRCNN-ds): neither PReLU nor `TRANSPOSE_CONV` has a fully supported int8 TFLM path on both target runtimes; replacing the 9×9 stride-2 deconvolution with a 3×3 convolution producing s^2 channels followed by `depth_to_space` preserves the learned-upsampling role at lower cost. *Residual-scale folding* (EDSR-Tiny): the 0.1 scaling applied to each residual branch is a multiplication by a constant, so after training we multiply the second convolution's kernel and bias of every block by 0.1 and remove the scaling op; the deployed graph is pure `CONV_2D`/`ADD`. For single-channel output, PyTorch's PixelShuffle and TensorFlow's `depth_to_space` coincide, so no channel reordering is needed.

B. Training and quantization pipeline

Models are defined, trained, and exported in one framework (*train-what-you-flash*): the exact graph that is flashed is the graph that was trained, which makes hybrid operators impossible by construction and removes any need for cross-framework weight porting and parity checks. Training uses Adam (10^{-3}, reduce-on-plateau), MAE loss, batch 32, 300 epochs, with the best checkpoint selected by validation PSNR. Post-training quantization follows the standard full-integer recipe [7]: per-channel int8 weights, per-tensor int8 activations calibrated on 200 training LR frames, and int8 input/output (`inference_input_type = inference_output_type =` int8). Every exported model is verified by walking its flatbuffer: the operator set must be exactly {`CONV_2D`, `ADD`, `DEPTH_TO_SPACE`} and all I/O tensors int8; the export fails otherwise.

IV. EXPERIMENTAL SETUP

A. Dataset and protocol

Quantitative evaluation uses grayscale pairs derived from high-resolution FLIR thermal captures[1]: each frame is whole-frame resized (area interpolation) to 64×48 ground truth and ×2 downsampled to a 32×24 input, giving 1238 training and 219 validation pairs. Whole-frame resizing, rather than texture crops, keeps the full-scene field-of-view statistics comparable to a native low-resolution thermal camera. Native MLX90640 recordings [8] are used *qualitatively only* (demo figures and the firmware's embedded test frame): no ground truth exists at 64×48 for them, so no PSNR is reported against pseudo-labels.

[1]FLIR-IISR frames, 1024×768; see the released pipeline for provenance.

B. Hardware platforms

ESP32-DevKitC V4 (dual-core Xtensa LX6 at 240 MHz, ∼520 KB SRAM, **no PSRAM**) [9] runs Espressif's TFLM fork with esp-nn kernels. The tensor arena is allocated from the internal heap at runtime; because the heap is fragmented by the runtime itself, the largest allocatable contiguous block is ∼110 KB, and the firmware ladders the requested arena size downward until allocation succeeds. *Raspberry Pi Pico* (RP2040, Cortex-M0+ at 133 MHz, 264 KB SRAM) [10] runs pico-tflmicro with a static arena. A single sketch per model targets both boards via conditional compilation and prints machine-parseable `CAS:key=value` lines: arena and heap sizes, first plus ten sampled `Invoke()` times, and a full int8 output dump that is compared against the PC TFLite interpreter output embedded at build time. Energy per frame is *estimated* as datasheet chip-level active current (50 mA ESP32, 25 mA RP2040, at 3.3 V) times measured latency; no power meter was used, and the figures exclude board regulators and USB.

V. RESULTS

A. Quality and quantization penalty

Table II reports validation quality. Every INT8 model beats the bicubic baseline (30.56 dB), by margins from +0.19 dB (ESPCN-Micro, 1.6 k parameters) to +2.02 dB (EDSR-Tiny). The quantization penalty is 0.17–0.64 dB and is *non-monotonic in model size*: ESPCN-Light loses 0.64 dB while both the smaller ESPCN-Micro (0.17 dB) and the larger ESPCN (0.28 dB) lose less. The penalty is a per-architecture sensitivity rather than a calibration artifact: increasing the calibration set from 200 to all 1238 training frames left it unchanged.

Fig. 1 compares the deployed INT8 models against the ground truth on two validation frames chosen by fixed rules (no cherry-picking): the frame at the 25th percentile of bicubic PSNR (a harder scene) and the frame with the most spatial detail (highest Laplacian variance). The models recover the silhouette edges and facade structure that bicubic blurs, and the per-frame PSNR ordering reproduces the validation-mean ranking of Table II.

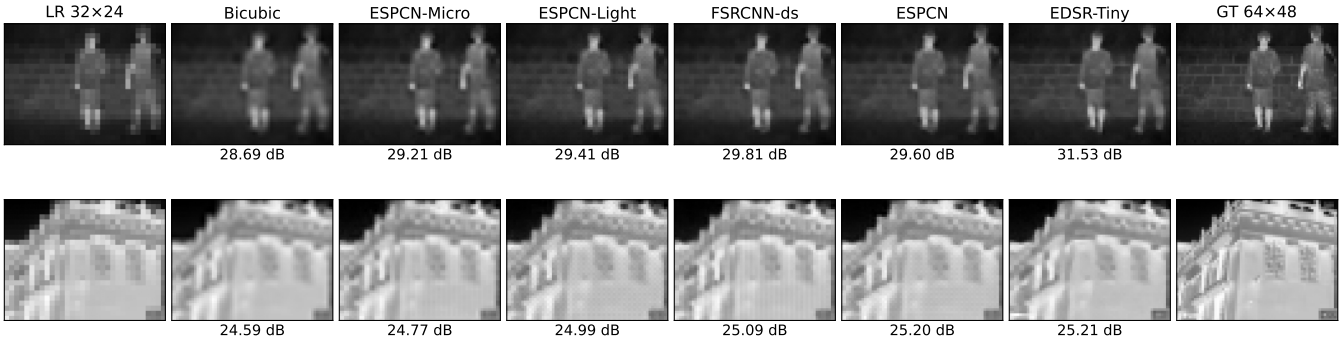

Fig. 1. FLIR validation frames super-resolved by the deployed INT8 models (LiteRT inference of the flashed flatbuffers), with per-frame PSNR against the 64×48 ground truth. Top: 25th-percentile-difficulty scene; bottom: highest-detail scene.

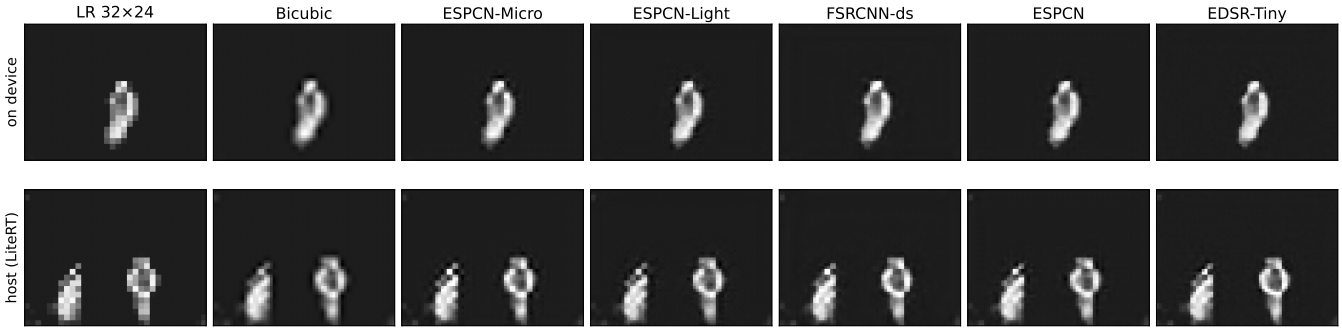

Fig. 2. Native MLX90640 frames (32×24) super-resolved to 64×48 (all INT8). *Top row:* the firmware's embedded test frame; the SR images shown are the exact int8 bytes captured from the boards' serial dumps (EDSR-Tiny from the Pico, all others from the ESP32). *Bottom row:* a further frame run on host with the same flashed flatbuffers (device-identical within the bounds of Sec. V-C). Qualitative only; no ground truth exists at the target resolution.

TABLE III
ON-DEVICE LATENCY, ARENA MEMORY AND ESTIMATED ENERGY PER FRAME.

	ESP32 (240 MHz)			Pico (133 MHz)		
Model	ms	arena KB	est. mJ	ms	arena KB	est. mJ
ESPCN-Micro	102.1	19.1	16.9	127.9	19.7	10.6
ESPCN-Light	337.9	37.2	55.8	402.6	38.5	33.2
FSRCNN-ds	628.0	54.0	103.6	717.4	54.2	59.2
ESPCN	969.1	73.6	159.9	1500.8	76.0	123.8
EDSR-Tiny	does not fit			10038.7	129.3	828.2

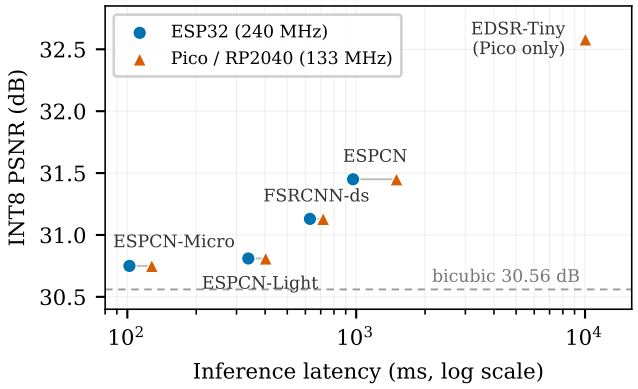

Fig. 3. Quality–latency trade-off of the INT8 models on both boards (log-scale latency).

B. On-device performance

Table III and Fig. 3 summarize the board measurements (mean of 10 `Invoke()` calls; the standard deviation is below 0.03 ms in all runs, so single measurements are representative).

The board measurements yield four observations. *(i) Real-time SR is feasible:* ESPCN-Micro runs at 9.8 FPS on the ESP32 and 7.8 FPS on the Pico, matching the MLX90640's typical 8 Hz refresh rate [1], at under 20 KB of arena. *(ii) The fit boundary is memory, not flash:* EDSR-Tiny (185 KB flash, 129 KB arena) runs on the Pico's 264 KB contiguous SRAM but cannot obtain a large-enough contiguous block on the ESP32's fragmented internal heap. This is a concrete, measured illustration that *usable* SRAM on the ESP32 is bounded by its largest free block (~110 KB), not by the 520 KB total. *(iii) The slower, simpler core is competitive:* despite a 1.8× clock deficit and no vector extensions, the RP2040 is only 14–26% slower than the ESP32 on the three smallest models (widening to 55% on ESPCN), and its estimated energy per frame is ~1.4–1.8× lower throughout. *(iv) Latency is*

predictable from the MAC count alone: across a 100× span in workload, the effective per-operation cost stays within a narrow band: 60–84 ns/MAC on the ESP32 and 83–105 ns/MAC on the RP2040. A designer can therefore estimate on-device latency of a candidate architecture to within ∼40% *before training it*, from the analytic MAC count of the graph. The most efficient point on the ESP32 is the full ESPCN (59.6 ns/MAC): its wide 64-channel convolutions amortize per-layer overhead best in the esp-nn kernels.

C. Device–host numerical fidelity

The firmware compares every output pixel against the PC TFLite interpreter reference. Outputs are not universally bit-exact, but the deviation is bounded and *identical on both boards*: the ESPCN family differs on at most 3.9% of pixels by ±1 LSB; FSRCNN-ds on 31% by up to ±4 LSB; EDSR-Tiny (Pico) on 49% by up to ±3 LSB. Equal mismatch counts on two unrelated cores indicate deterministic rounding differences in the optimized int8 kernels rather than platform defects, so PC-side quality metrics transfer to the device within these sub-quantization-step bounds.

VI. Discussion and Limitations

Choosing a model. The measurements suggest a simple selection rule. If the application must keep up with the sensor (8 Hz), ESPCN-Micro is currently the only option, and it still clears bicubic by +0.19 dB in 20 KB of RAM. If frames are processed on demand (e.g., a presence event triggers one high-quality readout), ESPCN at ∼1 s or EDSR-Tiny at ∼10 s (Pico) buy +0.9 dB and +2.0 dB respectively. FSRCNN-ds and ESPCN-Light are dominated in this benchmark: ESPCN reaches higher quality at comparable or better per-MAC efficiency, and their larger quantization penalties (0.49–0.64 dB) erode their float32 advantage.

Quantization penalty and QAT. The penalty is post-training quantization only; quantization-aware training [7] would likely recover much of ESPCN-Light's 0.64 dB, but was not applied here to keep every model on the identical, reproducible PTQ recipe. The penalty proved insensitive to calibration-set size (200 vs. all 1238 frames). This points to a genuine architecture sensitivity, plausibly the narrow 16-channel bottleneck concentrating activation ranges, rather than insufficient calibration statistics.

Protocol. The 64×48 ground truth is itself produced by downscaling high-resolution FLIR captures, so absolute PSNR values are specific to this protocol; the supported claims are the *relative* ranking of models and the quantization deltas. The on-device fidelity check uses one embedded test frame per model (flash budget), so its mismatch statistics are not a dataset-wide guarantee.

Energy. The energy column is a first-order estimate (datasheet chip-level active current × measured latency) that excludes board regulators, USB, and duty-cycling; it supports *relative* comparison between models and boards, not absolute power claims.

Headroom. Both targets are dual-core, yet TFLM executes single-threaded, leaving a near-2× latency reserve that could bring the mid-size models to sensor rate without touching the models themselves.

VII. Conclusion

Restricting compact SR networks to a three-operator, int8-native vocabulary makes full-integer quantization succeed by construction and costs at most 0.64 dB. The resulting models turn a 32×24 thermopile array into a 64×48 imager entirely on a low-cost MCU: at the small end, sensor-rate (8 Hz) super-resolution in 20 KB of RAM; at the large end, a +2 dB model that fits the Pico but is excluded from the ESP32 by heap fragmentation alone. Beyond the specific numbers, two engineering lessons generalize: full-integer deployability is an *architectural* property that should be designed in before training, and on-device latency of this model class is predictable from the analytic MAC count within tens of percent. We release the full pipeline (training, quantization with flatbuffer verification, dual-target firmware, log parsing) so every number is reproducible. Future work includes quantization-aware training to recover the ESPCN-Light penalty, dual-core execution, and true power measurements.

References

[1] Melexis, "MLX90640 32×24 IR array datasheet," https://www.melexis.com/en/product/MLX90640/, 2019.

[2] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[3] W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *European Conference on Computer Vision (ECCV)*, 2016.

[5] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *IEEE CVPR Workshops*, 2017.

[6] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a deep convolutional network for image super-resolution," in *European Conference on Computer Vision (ECCV)*, 2014.

[7] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[8] S. Zhu, T. Voigt, F. Rahimian, L. Mottola, and C. Perez-Penichet, "Infrared thermal dataset (MLX90640)," Zenodo record 5574233, 2021.

[9] Espressif Systems, "ESP32 series datasheet," https://www.espressif.com/, 2023.

[10] Raspberry Pi Ltd, "RP2040 datasheet," https://datasheets.raspberrypi.com/, 2023.